Project Documentation Report

Name: Satyajeet Dilip Zende

PRN: 2022011031229

College: D.Y.P.-A.T.U., Talsande.

Harnessing EC2, VPC, and MobaXterm for Cutting-Edge Entity Hosting

acerTIFY / EduBridge Project

Prepared: May 27, 2026

1. Executive Summary

This report documents the end-to-end deployment of acerTIFY (also referred to as the EduBridge Project), a highly available and securely hosted static web application provisioned on Amazon Web Services (AWS). The deployment integrates core cloud building blocks — Amazon Elastic Compute Cloud (EC2), Virtual Private Cloud (VPC), and Elastic Block Store (EBS) — with the Apache HTTP Server for content delivery, MobaXterm for hardened remote administration, and Amazon CloudWatch coupled with Simple Notification Service (SNS) for proactive monitoring and alerting.

The objective of the project was to demonstrate a production-style cloud workflow: provisioning isolated compute, securing administrative access through public-key cryptography, deploying a multi-page web application, validating connectivity in real-world network conditions, and instrumenting the workload for operational visibility. All goals were achieved, including the diagnosis and remediation of a real-world network egress restriction encountered during user-acceptance testing.

2. Project Objective

The goal of the project was to design, deploy, and validate a secure, highly available web application — acerTIFY — using native AWS infrastructure. The architecture is built around an EC2 instance launched inside a VPC for network isolation, MobaXterm for encrypted remote management, the Apache HTTP Server for web content delivery, and Amazon CloudWatch for real-time performance monitoring. The combined stack delivers a resilient foundation suitable for further application growth.

3. Architecture Overview

Compute Layer: A t2.micro EC2 instance running Amazon Linux serves as the application host. The instance is Free Tier eligible and is provisioned with 8 GiB of gp2 EBS storage for the root volume.

Networking Layer: The instance is launched into a VPC and exposed via a Public IPv4 address. A Security Group acts as a stateful virtual firewall, permitting only the protocols required for administration and web traffic.

Access Layer: Administrative access is performed exclusively over SSH using a private RSA key (.pem) generated at launch. MobaXterm acts as the local SSH client and terminal multiplexer.

Application Layer: Apache HTTP Server (httpd) serves a two-page static web application (index.html and about.html). The pages are styled using the Tailwind CSS CDN, eliminating the need for a local build pipeline.

Monitoring Layer: Amazon CloudWatch tracks the EC2 instance's CPUUtilization metric. An alarm is wired to an Amazon SNS topic that dispatches email notifications to the administrator when defined thresholds are breached.

3.1 Inbound Security Group Rules

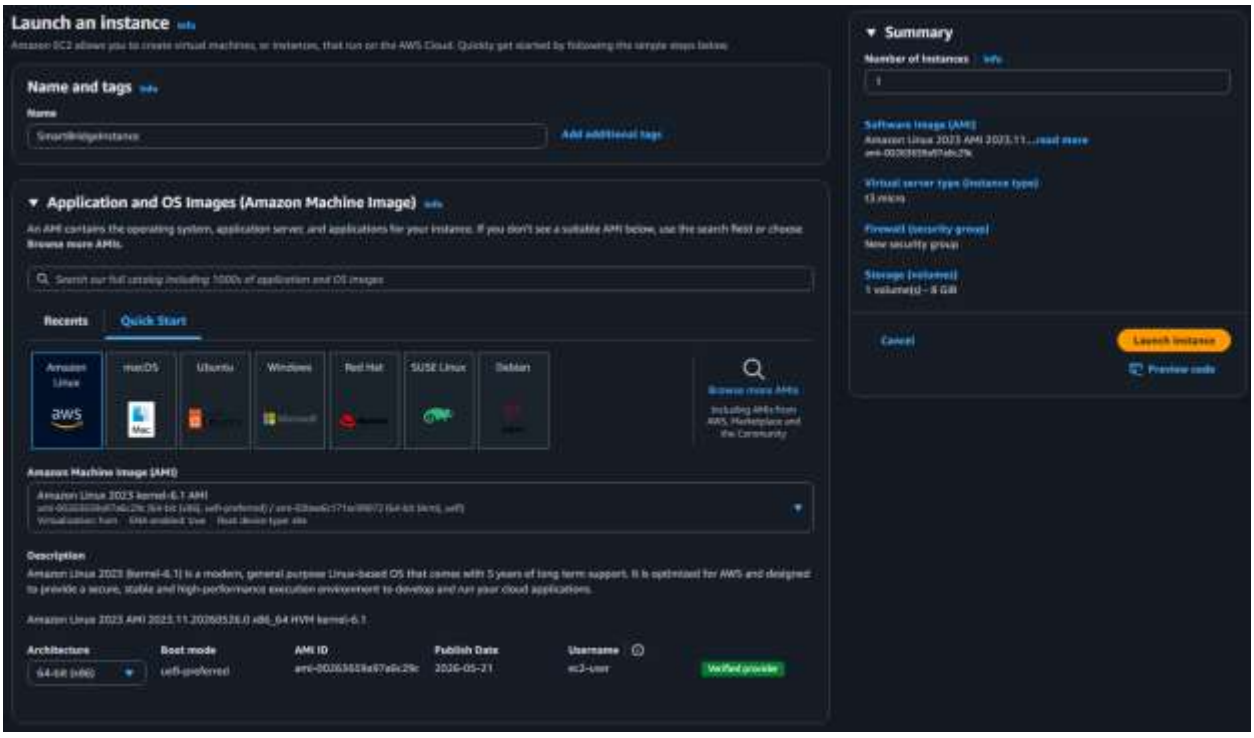
Protocol	Port	Source	Purpose
SSH	22	0.0.0.0/0	Encrypted remote administration via MobaXterm
HTTP	80	0.0.0.0/0	Public web traffic to Apache
HTTPS	443	0.0.0.0/0	Reserved for future TLS-encrypted web traffic

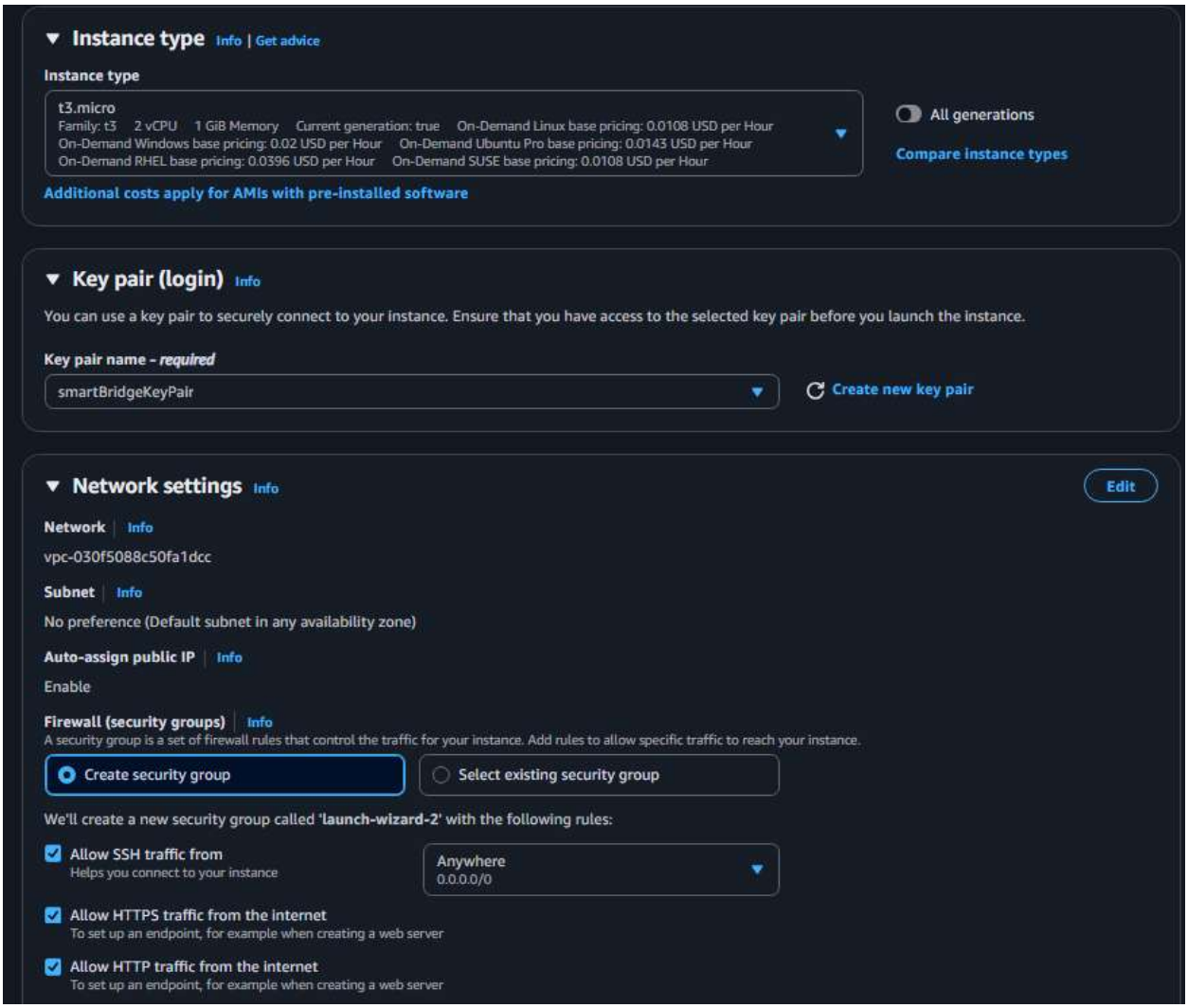
4. Implementation Steps

Phase 1: AWS Infrastructure Provisioning

- Signed in to the AWS Management Console and opened the EC2 launch wizard.

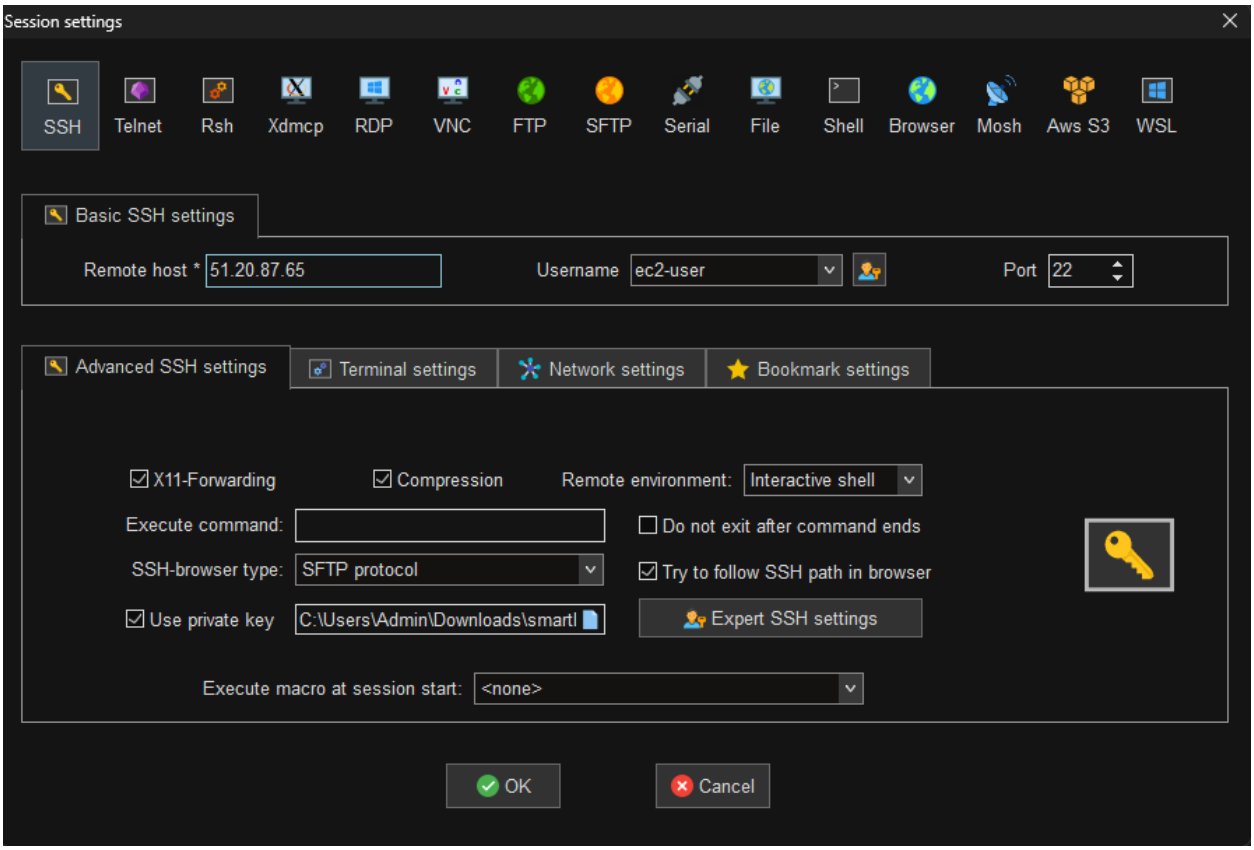
- Selected the Amazon Linux AMI (Free Tier eligible) as the base operating system.
- Configured compute capacity using the t2.micro instance type.
- Generated a new RSA key pair and securely downloaded the .pem private key for SSH access.
- Created an inbound Security Group rule set: SSH (22), HTTP (80), and HTTPS (443) — each opened to 0.0.0.0/0 to allow global administrative and web access.
- Provisioned an 8 GiB gp2 EBS volume as the root storage device.




Instance type Info | Get advice
Instance type
t3.micro
Family: t3 2 vCPU 1 GiB Memory Current generation: true On-Demand Linux base pricing: 0.0108 USD per Hour
On-Demand Windows base pricing: 0.02 USD per Hour On-Demand Ubuntu Pro base pricing: 0.0143 USD per Hour
On-Demand RHEL base pricing: 0.0396 USD per Hour On-Demand SUSE base pricing: 0.0108 USD per Hour
All generations
Compare instance types
Additional costs apply for AMIs with pre-installed software
Key pair (login) Info
You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.
Key pair name - required
smartBridgeKeyPair
Create new key pair
Network settings Info
Edit
Network | Info
vpc-030f5088c50fa1dcc
Subnet | Info
No preference (Default subnet in any availability zone)
Auto-assign public IP | Info
Enable
Firewall (security groups) | Info
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.
Create security group
Select existing security group
We'll create a new security group called 'launch-wizard-2' with the following rules:
Allow SSH traffic from
Helps you connect to your instance
Anywhere
0.0.0.0/0
Allow HTTPS traffic from the internet
To set up an endpoint, for example when creating a web server
Allow HTTP traffic from the internet
To set up an endpoint, for example when creating a web server

Phase 2: Secure Remote Management via MobaXterm

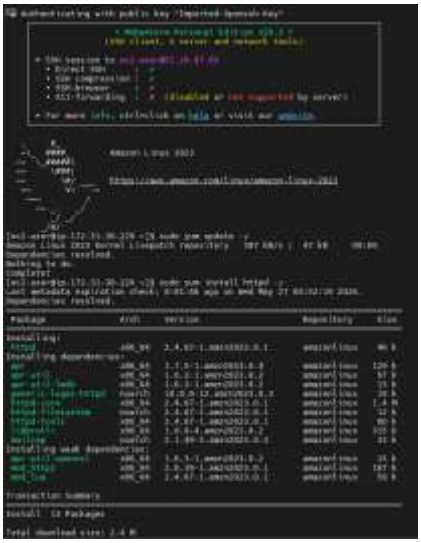
- Configured a new SSH session in MobaXterm targeting the EC2 instance's Public IPv4 address.
- Defined the default operating system user as ec2-user.
- Attached the downloaded .pem private key under Advanced SSH settings, replacing password authentication with public-key cryptography.
- Established the terminal session and confirmed shell access to the remote AWS environment.

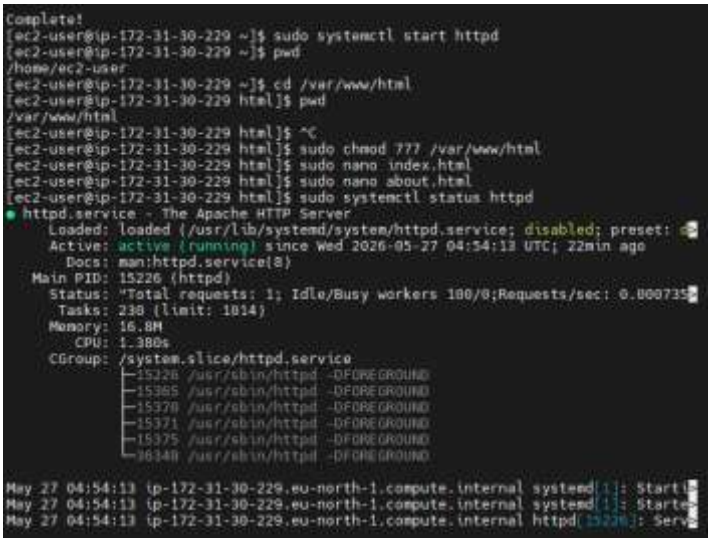


Phase 3: Web Server Installation and Configuration

The following commands were executed sequentially on the EC2 instance:

- sudo yum update -y — patched the system and refreshed all package lists.
- sudo yum install httpd -y — installed the Apache HTTP Server.
- sudo systemctl start httpd — started the Apache daemon.
- sudo chmod 777 /var/www/html — opened directory permissions to allow application deployment into the Apache document root.

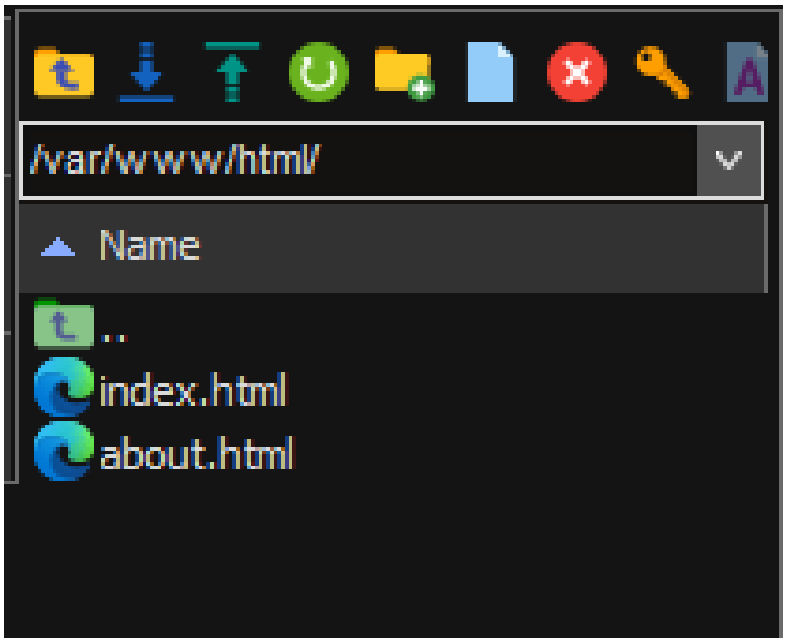




Phase 4: Application Deployment

Rather than relying on SFTP-based file transfers, the application was authored directly on the instance using the nano CLI text editor inside /var/www/html. This streamlined the workflow and removed dependencies on additional transfer tooling.

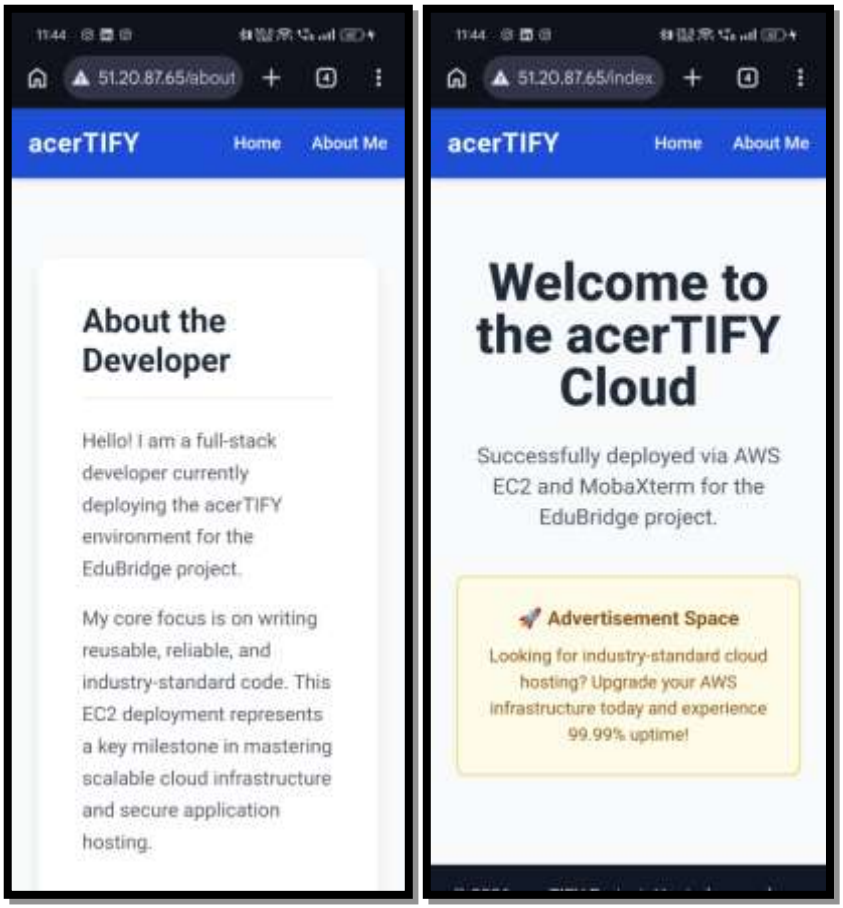
- Engineered a two-page static web application consisting of index.html and about.html.
- Used the Tailwind CSS CDN inside the HTML <head> to deliver a modern, responsive interface without a local build pipeline.
- Simulated an advertisement space and an “About the Developer” section to validate multi-page routing on the Apache server.



Phase 5: Testing and Troubleshooting

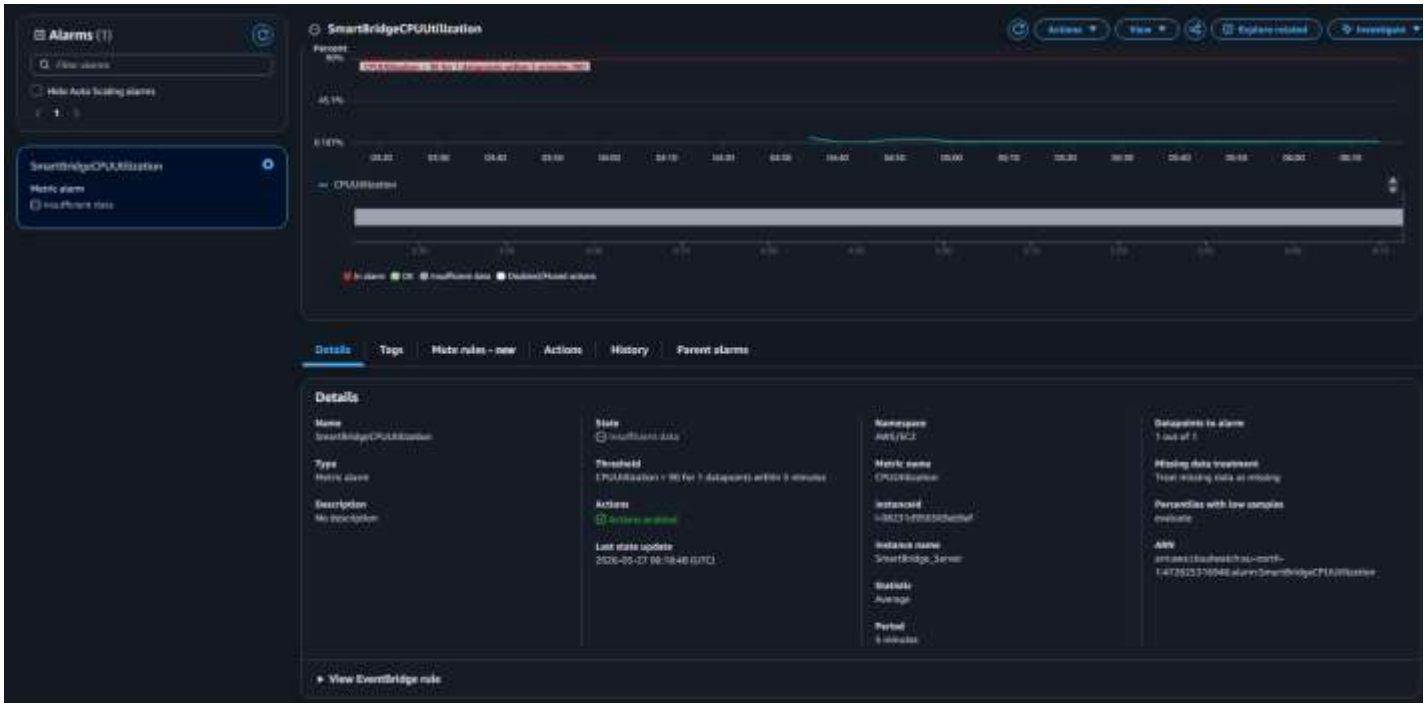
Initial browser testing against the EC2 Public IP produced a “Site Unreachable” error. Methodical diagnosis isolated the fault to the client network, not the server stack.

- Ran curl http://localhost on the EC2 instance via MobaXterm — the HTML payload was returned successfully, confirming Apache was healthy and serving content.
- Identified the root cause as a corporate/university network firewall aggressively dropping unencrypted outbound traffic on Port 80.
- Bypassed the restriction by switching the client machine to an unrestricted mobile hotspot, immediately bringing the website live.



Phase 6: Monitoring and Optimization

- Implemented real-time infrastructure monitoring using Amazon CloudWatch.
- Created a proactive alarm targeting the EC2 instance's CPUUtilization metric.
- Configured a static threshold of ≥ 90% CPU utilization to trigger the "In Alarm" state.
- Connected the alarm to an Amazon SNS topic configured to dispatch an automated email alert to the administrator, enabling rapid response and dynamic resource management.



5. Troubleshooting Summary

Symptom	Diagnosis	Resolution
Browser shows “Site Unreachable” when visiting EC2 Public IP.	curl http://localhost on the instance succeeds — Apache is healthy. Outbound Port 80 traffic from the client network is being dropped.	Switch the client machine to an unrestricted network (mobile hotspot). For long-term remediation, enable HTTPS on Port 443 with a TLS certificate.
SSH login prompts for a password.	MobaXterm session was not configured with the private key.	Attach the .pem key under MobaXterm’s Advanced SSH settings to enable key-based authentication.
Unable to write files into /var/www/html.	Default directory permissions prevent the ec2-user from writing into the Apache document root.	Apply sudo chmod 777 /var/www/html during initial setup (tighten permissions for production deployments).

6. Security Considerations

- Public-key authentication via a downloaded RSA .pem key replaces password-based SSH, eliminating brute-force exposure.
- Security Group rules implement least-privilege ingress: only Ports 22, 80, and 443 are open.
- chmod 777 on /var/www/html is acceptable for demonstration but should be tightened (e.g. 755 with proper ownership) before production use.
- HTTPS (Port 443) is pre-authorized in the Security Group, allowing immediate TLS rollout via a future ACM or Let’s Encrypt certificate.

7. Conclusion

This deployment demonstrates a sophisticated, real-world approach to modern web hosting. By integrating AWS compute resources (EC2, VPC, EBS), efficient remote management tooling (MobaXterm), a battle-tested web server (Apache), and proactive observability (CloudWatch + SNS), the acerTIFY project delivers an architecture optimized for performance, scalability, and security. The troubleshooting episode encountered during Phase 5 further reinforced the importance of distinguishing between server-side and client-side network failures — a skill that translates directly into production operations. The resulting platform provides a resilient foundation upon which future iterations of the acerTIFY / EduBridge application can confidently grow.